KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

uv로 파이썬 시작하기

예전 방식과 비교하고,
자주 쓰는 명령 익히기

```
$ uv init hello-uv
  프로젝트 만들기
$ uv add fastapi
  패키지 넣기
$ uv run main.py
  실행하기
$ uv sync
  다른 컴퓨터에서 그대로
```

오늘의 순서

1 **파이썬은 어떻게 실행되나**
python, 패키지
3–4쪽

2 **예전 방식**
venv, pip, requirements.txt
5–8쪽

3 **uv는 무엇이 다른가**
비교와 설치
9–12쪽

4 **프로젝트 만들고 실행하기**
uv init, uv run, 가상환경 켜기
13–18쪽

5 **패키지 넣고 빼기**
uv add, uv tree, uv remove
19–23쪽

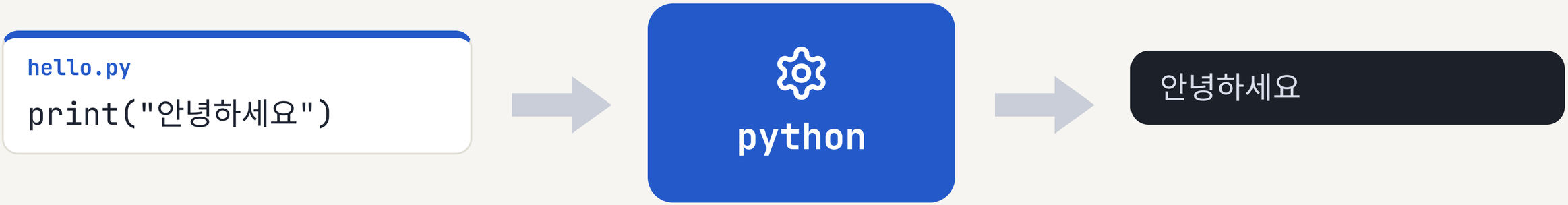
6 **다른 컴퓨터에서 그대로**
GitHub, uv sync
24–27쪽

7 **VS Code에서 Jupyter 쓰기**
확장 설치, ipykernel, 커널 고르기
28–33쪽

34–36쪽은 명령 바꿔 치기, 막힐 때, 한 장 요약이에요.

1. 파이썬은 어떻게 실행되나

파이썬 코드는 python이 읽고 실행해요




```
$ python3 hello.py
안녕하세요
```

.py 파일은 글자일 뿐이에요. python 프로그램이 한 줄씩 읽어서 실행해요.

1. 파이썬은 어떻게 실행되나

남이 만든 코드 묶음을 패키지라고 해요

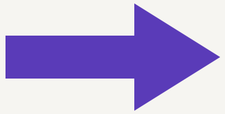
PyPI 파이썬 패키지 창고

`fastapi` `uvicorn` `pandas` `numpy`

`openai` `requests` `pytest`

`… 60만 개 넘게`

받아서 설치



내 프로젝트/

`main.py`

`fastapi` `uvicorn`

fastapi로 웹 서버 코드를 쓰고,
uvicorn으로 그 서버를 켜요.

이 자료는 fastapi와 uvicorn을 넣고, 쓰고, 빼 봐요.

2. 예전 방식

예전에는 이 순서를 직접 챙겼어요

1	파이썬 설치	python.org에서 받아 설치	python
2	가상환경 만들기	`python3 -m venv .venv`	venv
3	가상환경 켜기	`source .venv/bin/activate`	venv
4	패키지 설치	`pip install fastapi uvicorn`	pip
5	목록 적어 두기	`pip freeze > requirements.txt`	requirements.txt
6	실행	`python main.py`	python

2. 예전 방식

가상환경은 프로젝트마다 따로 쓰는 패키지 상자예요

컴퓨터에 설치한 파이썬 여기에 바로 넣으면 모든 프로젝트가 같이 써요

`project-a/`

`main.py`

`.venv/`

`fastapi 0.162.0`

`project-b/`

`main.py`

`.venv/`

`fastapi 0.110.0`

프로젝트끼리 패키지 버전이 달라도 부딪히지 않아요. uv도 이 .venv를 그대로 써요.

2. 예전 방식

예전 방식을 실제로 쳐 보면

```
$ python3 main.py
ModuleNotFoundError: No module named 'fastapi'
$ python3 -m venv .venv
$ source .venv/bin/activate
(.venv) $ pip install fastapi uvicorn
Collecting fastapi
  ... 중간 줄 생략 ...
Successfully installed annotated-doc-0.0.5 annotated-types-0.8.0 anyio-4.15.1 click-8.5.0 …
(.venv) $ pip freeze > requirements.txt
(.venv) $ python main.py
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

가상환경을 켜면 프롬프트 앞에 (.venv)가 붙어요. 켠 상태에서만 패키지가 보여요.

2. 예전 방식

예전 방식에서 자주 막히는 곳

켜는 걸 잊으면

deactivate 뒤에 python3 main.py

ModuleNotFoundError

목록은 손으로

pip install 뒤 pip freeze를 잊으면

requirements.txt가 옛날 그대로

빼도 남아요

pip uninstall fastapi uvicorn 뒤에도

pydantic, starlette 같은 12개가 남아요

파이썬은 따로

설치와 버전 맞추기는 직접

python, python3 이름도 컴퓨터마다 달라요

3. uv는 무엇이 다른가

예전에는 일마다 도구가 달랐어요

파이썬 설치

`python.org`

가상환경

`venv, activate`

패키지 설치

`pip`

목록 적기

`requirements.txt`

네 가지를 각각 배우고, 순서도 직접 기억해야 했어요.

3. uv는 무엇이 다른가

uv 하나가 네 가지를 다 해요

UV

파이썬 설치

필요하면 알아서 받아요

가상환경

uv run이 만들고 켜요

패키지 설치

uv add

목록 적기

uv.lock에 자동으로

명령은 uv 하나로 시작해요. 가상환경을 켜고 끄는 일은 uv run이 대신해요.

3. uv는 무엇이 다른가

할 일마다 비교해 봐요

할 일	예전	uv
파이썬 설치	python.org에서 받아 직접 설치	**없으면 uv가 받아요**
가상환경	python3 -m venv .venv 뒤 activate	`uv run`이 알아서 만들고 켜요
패키지 넣기	pip install, 그다음 pip freeze	`uv add fastapi uvicorn`
패키지 빼기	pip uninstall, 딸린 패키지는 남아요	`uv remove fastapi uvicorn`
다른 컴퓨터	pip install -r requirements.txt	`uv sync`
실행	켠 상태에서 python main.py	`uv run main.py`

uv 설치하기

macOS, Linux 터미널에 붙여 넣기

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows PowerShell에 붙여 넣기

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

3 터미널을 닫았다가 새로 열어요. Windows는 Git Bash를 새로 열어요.

```
$ uv --version
uv 0.11.32 (x86_64-unknown-linux-gnu)
```

4. 프로젝트 만들고 실행하기

uv init으로 프로젝트를 만들어요

```
$ cd ~
$ uv init hello-uv
Downloading cpython-3.14.6-linux-x86_64-gnu (download) (34.3MiB)
 Downloaded cpython-3.14.6-linux-x86_64-gnu (download)
Initialized project `hello-uv` at `/home/kim/hello-uv`
$ cd hello-uv
$ ls -a
```

hello-uv/

.git	Git 저장소 (02)
.gitignore	Git에 안 올릴 목록
.python-version	쓸 파이썬 버전
README.md	프로젝트 소개 글
main.py	실행할 코드
pyproject.toml	프로젝트 설명서

컴퓨터에 파이썬이 없으면 uv가 받아서 써요. 위 두 줄이 그 표시예요.

pyproject.toml은 프로젝트 설명서예요

pyproject.toml

```
[project]
name = "hello-uv"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.14"
dependencies = []
```

◀ 프로젝트 이름

◀ 쓸 수 있는 파이썬 버전

◀ 쓰는 패키지, 아직 없어요

.python-version

```
3.14
```

main.py

```
def main():
    print("Hello from hello-uv!")


if __name__ == "__main__":
    main()
```

4. 프로젝트 만들고 실행하기

uv run으로 실행해요

```
$ uv run main.py
Using CPython 3.14.6
Creating virtual environment at: .venv
Hello from hello-uv!
```

hello-uv/

.git	Git 저장소 (02)
.gitignore	Git에 안 올릴 목록
.python-version	쓸 파이썬 버전
.venv	가상환경 (패키지 상자)
README.md	프로젝트 소개 글
main.py	실행할 코드
pyproject.toml	프로젝트 설명서
uv.lock	정확한 버전 목록

처음 한 번 .venv와 uv.lock을 만들어요. activate는 치지 않아요.

4. 프로젝트 만들고 실행하기

두 번째부터는 바로 실행돼요

```
$ uv run main.py
Hello from hello-uv!
```



예전

```
source .venv/bin/activate
python main.py
```

uv

```
uv run main.py
```

uv run은 켜고, 실행하고, 끄는 일을 한 번에 해요.

4. 프로젝트 만들고 실행하기

가상환경을 직접 켤 수도 있어요

macOS, Linux 터미널

```
source .venv/bin/activate
```

Windows Git Bash

```
source .venv/Scripts/activate
```

Windows PowerShell

```
.venv\Scripts\activate
```

끄기 모든 터미널

```
deactivate
```

```
$ source .venv/bin/activate
(hello-uv) $ which python
/home/kim/hello-uv/.venv/bin/python
(hello-uv) $ python --version
Python 3.14.6
(hello-uv) $ python main.py
Hello from hello-uv!
(hello-uv) $ deactivate
$
```

켜면 프롬프트 앞에 (hello-uv)가 붙고, python으로 바로 실행돼요. 수업은 uv run을 기본으로 써요.

파이썬 버전은 .python-version이 정해요

```
$ cd ~
$ uv init --python 3.13 hello-313
Initialized project `hello-313` at `/home/kim/hello-313`
$ cd hello-313
$ cat .python-version
3.13
$ uv run python --version
Downloading cpython-3.13.14-linux-x86_64-gnu (download) (33.2MiB)
 Downloaded cpython-3.13.14-linux-x86_64-gnu (download)
Using CPython 3.13.14
Creating virtual environment at: .venv
Python 3.13.14
```

버전을 정해서 만들려면 --python을 붙여요. 없는 버전은 uv가 받아요.

5. 패키지 넣고 빼기

uv add로 패키지를 넣어요

```
$ uv add fastapi uvicorn
Resolved 15 packages in 1.59s
Downloading pydantic-core (2.1MiB)
 Downloaded pydantic-core
Prepared 14 packages in 105ms
Installed 14 packages in 6ms
 + annotated-doc==0.0.5
 + annotated-types==0.8.0
 + anyio==4.15.1
 + click==8.5.0
 + fastapi==0.162.0
 + h11==0.16.0
 + idna==3.20
 + opentelemetry-api==1.45.2
 + pydantic==2.14.2
 + pydantic-core==2.50.2
 + starlette==1.10.0
 + typing-extensions==4.16.0
 + typing-inspection==0.4.4
 + uvicorn==0.54.0
```

pyproject.toml 내가 고른 패키지

```
"fastapi>=0.162.0",
"uvicorn>=0.54.0",
```

uv.lock 따라온 것까지 정확한 버전

고른 2개와 따라온 12개

pydantic 2.14.2, starlette 1.10.0 …

.venv 실제로 설치된 패키지

패키지 14개 설치

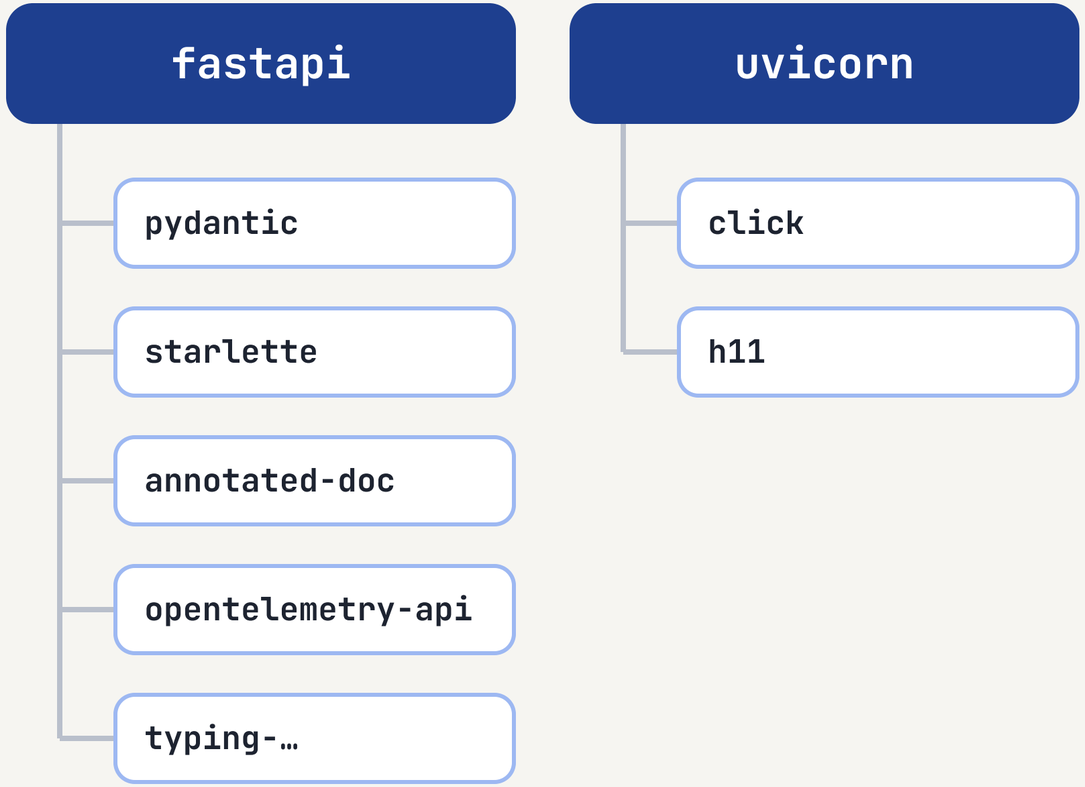
uv run으로 실행하면 바로 써요

명령 하나로 세 곳이 함께 바뀌어요.

5. 패키지 넣고 빼기

두 개를 넣었는데 14개가 들어왔어요

```
$ uv tree --depth 2
Resolved 15 packages in 1ms
hello-uv v0.1.0
├── fastapi v0.162.0
│   ├── annotated-doc v0.0.5
│   ├── opentelemetry-api v1.45.2
│   ├── pydantic v2.14.2
│   ├── starlette v1.10.0
│   ├── typing-extensions v4.16.0
│   └── typing-inspection v0.4.4
└── uvicorn v0.54.0
    ├── click v8.5.0
    └── h11 v0.16.0
```



fastapi와 uvicorn이 쓰는 패키지까지 uv가 함께 넣어요. 뺄 때도 함께 빠져요.

5. 패키지 넣고 빼기

넣은 패키지로 웹 서버를 켜요

main.py

```python
from fastapi import FastAPI
import uvicorn

app = FastAPI()


@app.get("/")
def home():
    return {"message": "안녕하세요"}


if __name__ == "__main__":
    uvicorn.run(app)
```

http://127.0.0.1:8000

{"message":"안녕하세요"}

브라우저 주소창에 http://127.0.0.1:8000

```
$ uv run main.py
INFO:     Started server process [448]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     127.0.0.1:45266 - "GET / HTTP/1.1" 200 OK
```

서버는 Ctrl+C로 끌 때까지 켜져 있어요.

5. 패키지 넣고 빼기

uv remove로 패키지를 빼요

```
$ uv remove fastapi uvicorn
Resolved 1 package in 2ms
Uninstalled 14 packages in 19ms
 - annotated-doc==0.0.5
 - annotated-types==0.8.0
 - anyio==4.15.1
 - click==8.5.0
   ... 9줄 생략 ...
 - uvicorn==0.54.0
$ uv run main.py
ModuleNotFoundError: No module named 'fastapi'
```

pyproject.toml 내가 고른 패키지

dependencies = []

uv.lock 따라온 것까지 정확한 버전

프로젝트만 남아요

따라왔던 12개도 지워요

.venv 실제로 설치된 패키지

패키지 14개 삭제

딸린 패키지까지 같이 빠져요

뺄 때도 세 곳이 함께 바뀌어요. 예전 pip uninstall은 12개가 남았어요.

5. 패키지 넣고 빼기

다시 넣으면 바로 들어가요

```
$ uv add fastapi uvicorn
Resolved 15 packages in 9ms
Installed 14 packages in 4ms
 + annotated-doc==0.0.5
 + annotated-types==0.8.0
 + anyio==4.15.1
 + click==8.5.0
   ... 9줄 생략 ...
 + uvicorn==0.54.0
$ uv run main.py
INFO:     Uvicorn running on http://127.0.0.1:8000
```

uv 캐시

한 번 받은 패키지를 컴퓨터에 보관해요.
다른 프로젝트에서도 다시 받지 않아요.

처음 넣을 때 있던 줄

`Prepared 14 packages in 105ms`

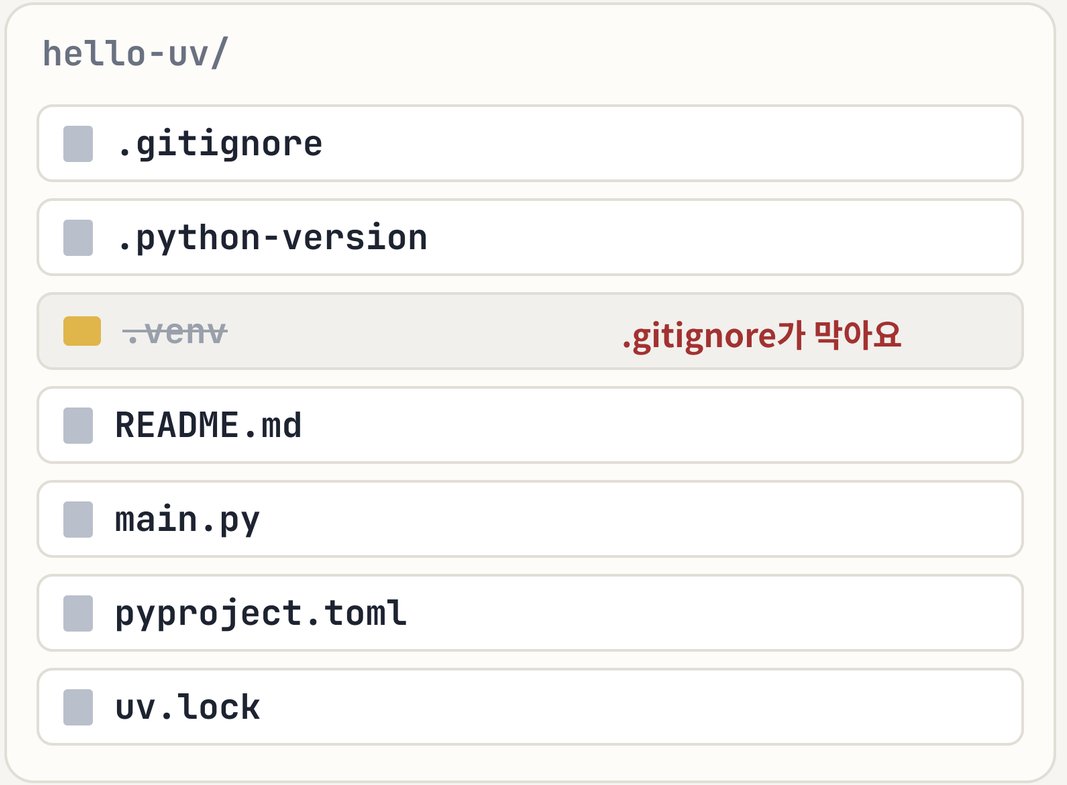
이번엔 받을 게 없어서 이 줄이 없어요.

uv가 빠른 이유 중 하나예요. 패키지 14개가 0.004초 만에 들어갔어요.

6. 다른 컴퓨터에서 그대로

GitHub에는 무엇을 올리나요

```
$ git status --short
?? .gitignore
?? .python-version
?? README.md
?? main.py
?? pyproject.toml
?? uv.lock
$ git add .
$ git commit -m "feat: hello-uv 프로젝트 만들기"
[main (root-commit) 12cad3c] feat: hello-uv 프로젝트 만들기
```



.venv는 올리지 않아요. pyproject.toml과 uv.lock만 있으면 똑같이 다시 만들어요.

6. 다른 컴퓨터에서 그대로

lee 컴퓨터에서는 uv sync 한 번이면 돼요



uv.lock에 적힌 버전 그대로 설치돼서 kim과 lee의 환경이 같아요.

6. 다른 컴퓨터에서 그대로

lee 컴퓨터에서 실제로 쳐 보면

```
$ git clone https://github.com/kim/hello-uv.git
Cloning into 'hello-uv'...
$ cd hello-uv
$ uv sync
Downloading cpython-3.14.6-linux-x86_64-gnu (download) (34.3MiB)
 Downloaded cpython-3.14.6-linux-x86_64-gnu (download)
Using CPython 3.14.6
Creating virtual environment at: .venv
Resolved 15 packages in 1ms
Prepared 14 packages in 85ms
Installed 14 packages in 4ms
 + annotated-doc==0.0.5  + annotated-types==0.8.0  + anyio==4.15.1  …
$ uv run main.py
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

6. 다른 컴퓨터에서 그대로

uv sync는 이럴 때 쳐요

1

clone한 직후

```
git clone 주소
```

```
cd 폴더
```

```
uv sync
```

2

pull로 uv.lock이 바뀐 뒤

```
git pull origin main
```

```
uv sync
```

3

uv run도 먼저 맞춰요

```
uv run main.py
```

실행 전에 sync를 알아서 해요

팀원이 패키지를 넣고 push했으면, pull 뒤 uv sync로 내 .venv를 맞춰요.

7. VS Code에서 Jupyter 쓰기

노트북은 코드를 셀 단위로 실행해요

main.py

```
데이터 읽기
정리하기
그래프 그리기
```

처음부터 끝까지 한 번에 실행해요.
결과는 터미널에 모여서 나와요.

notebook.ipynb

[1] 데이터 읽기

표 5줄

[2] 정리하기

빈칸 3개 지움

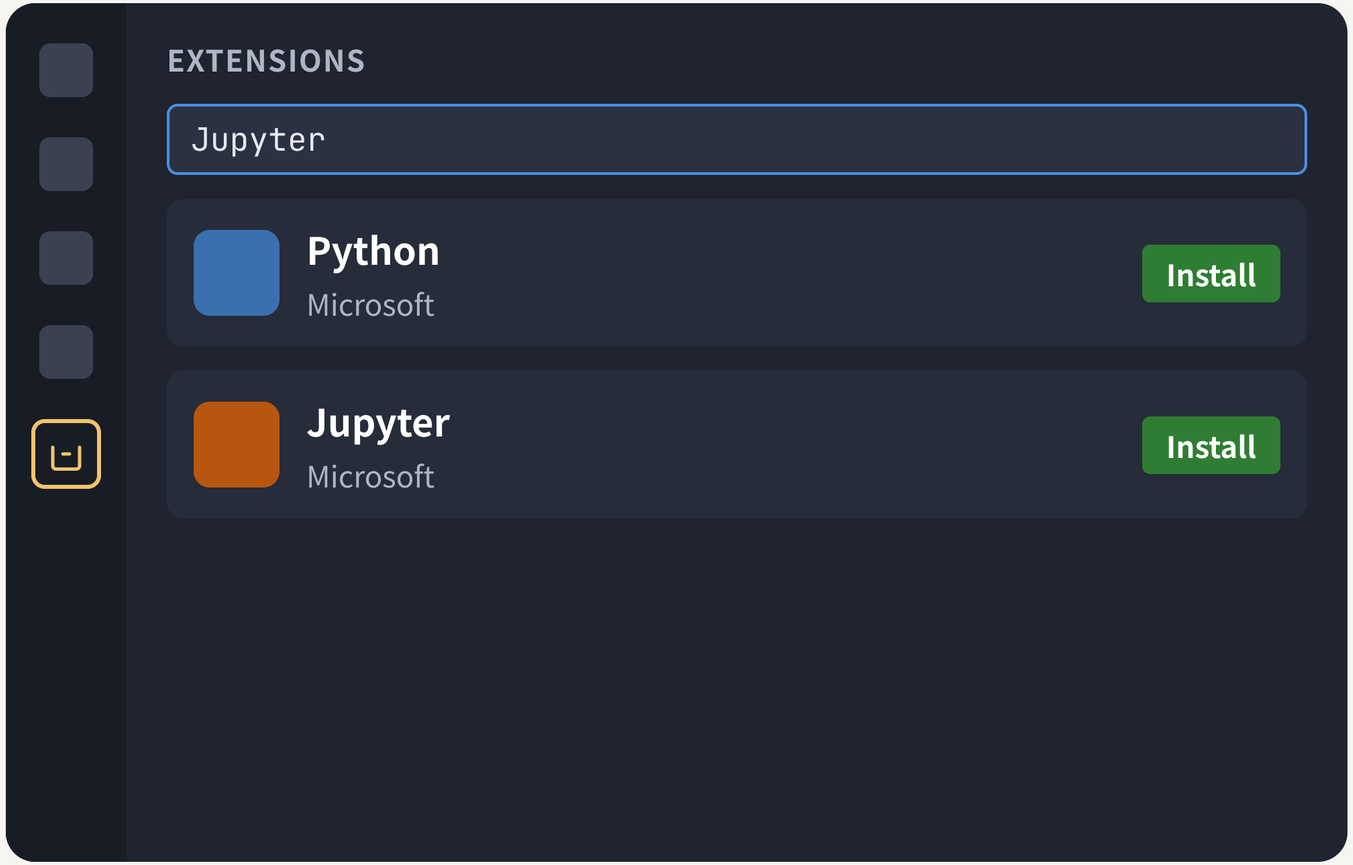
[3] 그래프 그리기

그래프 그림

셀 하나씩 실행하고 결과를 바로 아래에서 봐요. 머신러닝 수업에서 데이터를 조금씩 확인할 때 써요.

7. VS Code에서 Jupyter 쓰기

VS Code에 확장 두 개를 설치해요



1 **왼쪽 확장 아이콘을 눌러요**
Mac Cmd+Shift+X, Windows Ctrl+Shift+X

2 **Python을 검색해 설치해요**
만든 곳이 Microsoft인지 봐요

3 **Jupyter를 검색해 설치해요**
같이 깔리는 확장은 그대로 둬요

컴퓨터마다 한 번만 설치해요. 설치한 뒤에는 VS Code를 다시 열어요.

uv add --dev ipykernel로 커널을 넣어요

```
$ cd ~/hello-uv
$ uv add --dev ipykernel
Resolved 48 packages in 248ms
   ... Downloading 줄 생략 ...
Prepared 29 packages in 1.26s
Installed 29 packages in 68ms
 + asttokens==3.0.2
 + comm==0.2.3
   ... 26줄 생략 ...
 + wcwidth==0.9.4
```

pyproject.toml

```
dependencies = [
    "fastapi>=0.162.0",
    "uvicorn>=0.54.0",
]

[dependency-groups]
dev = [
    "ipykernel>=7.4.0",
]
```

커널 = 노트북 셀을 실행하는 파이썬

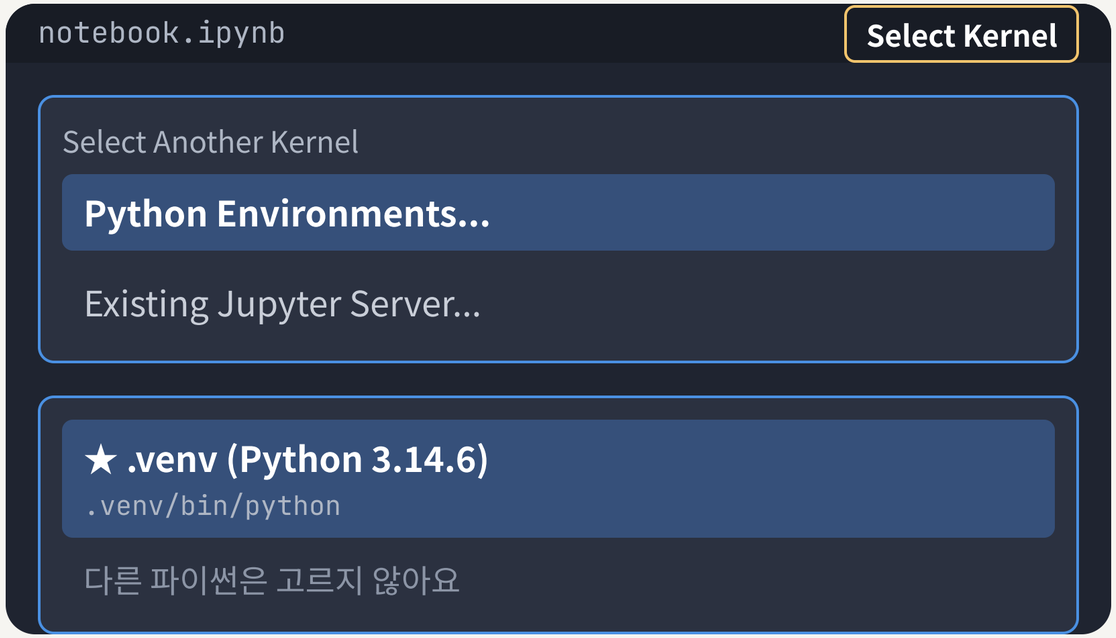
--dev는 개발할 때만 쓰는 도구라는 표시예요.
서버를 켜는 데는 필요 없어서 따로 적혀요.

노트북도 .venv 안의 파이썬으로 실행해요. 그래서 ipykernel을 .venv에 넣어요.

7. VS Code에서 Jupyter 쓰기

노트북을 만들고 커널로 .venv를 골라요

1. **File > Open Folder로 hello-uv 열기**
   폴더 자체를 열어요
2. **왼쪽 탐색기에서 새 파일 notebook.ipynb**
   이름 끝이 .ipynb면 노트북으로 열려요
3. **오른쪽 위 Select Kernel 누르기**
4. **Python Environments 고르기**
5. **.venv (Python 3.14.6) 고르기**
   hello-uv 안의 .venv예요



커널을 한 번 고르면 이 노트북을 열 때마다 기억해요.

7. VS Code에서 Jupyter 쓰기

Shift+Enter로 셀을 실행해요

notebook.ipynb .venv (Python 3.14.6)

[1]
```
print("안녕하세요")
```
안녕하세요

[2]
```
import sys
print(sys.executable)
```
/home/kim/hello-uv/.venv/bin/python

[3]
```
import fastapi
print(fastapi.__version__)
```
0.162.0

[4]
```
1 + 2
```
3

2번 셀에 .venv가 보이면 커널을 잘 고른 거예요. uv add로 넣은 패키지도 바로 써요.

7. VS Code에서 Jupyter 쓰기

노트북에서 막히면

1

ipykernel을 설치하라는 알림이 떠요

알림의 Install은 pip로 넣어서 pyproject.toml에 안 적혀요

```
uv add --dev ipykernel
```

2

커널 목록에 .venv가 없어요

hello-uv 폴더 자체를 열었는지 봐요. 그다음 VS Code 다시 열기

```
uv sync
```

3

셀에서 ModuleNotFoundError

커널이 .venv가 아니에요. 오른쪽 위 커널 이름을 눌러 바꿔요

```
.venv (Python 3.14.6)
```

정리

예전 명령은 이렇게 바뀌었어요

`python.org에서 설치`	→	없으면 uv가 받아요
`python3 -m venv .venv`	→	uv run, uv sync가 만들어요
`source .venv/bin/activate`	→	uv run이 대신해요. 켜도 돼요
`pip install fastapi`	→	`uv add fastapi`
`pip uninstall fastapi`	→	`uv remove fastapi`
`pip install -r requirements.txt`	→	`uv sync`
`python main.py`	→	`uv run main.py`

정리

이런 메시지가 나와요

1 `bash: uv: command not found`
설치 뒤 터미널을 새로 열지 않았어요

터미널 닫고 다시 열기

2 `error: No `pyproject.toml` found in current directory or any parent directory`
프로젝트 폴더 밖에서 쳤어요

```
cd hello-uv
```

3 `ModuleNotFoundError: No module named 'fastapi'`
패키지를 안 넣었거나 python3로 실행했어요

```
uv add fastapi uvicorn
uv run main.py
```

4 `error: Project is already initialized in `...``
이미 만든 프로젝트 안에서 uv init을 쳤어요

uv init은 처음 한 번만

한 장 요약

`uv init 이름`

프로젝트 폴더 만들기

`uv run main.py`

가상환경에서 실행

`uv add 패키지`

패키지 넣기

`uv remove 패키지`

패키지 빼기

`uv sync`

uv.lock대로 맞추기

`uv tree`

패키지 관계 보기

GitHub에 올려요 `pyproject.toml uv.lock .python-version`

올리지 않아요 `.venv` uv init이 .gitignore에 적어 둬요

노트북 `uv add --dev ipykernel` 뒤 커널로 .venv 고르기